Lecture 4

From Word Embedding to Contextual Embedding using RNN, LSTM, and GRU

Lecture Notes

MAI554 – Deep Learning for NLP

Prof. Anis Koubaa

College of Engineering, Alfaisal University

akoubaa@alfaisal.edu

Spring 2025

How to Use These Notes

These notes distill the key ideas from Lecture 4 into a logical flow. Each section follows a pattern: **intuition first, then math, then a concrete example**. Self-check questions appear throughout the text. Full solutions are collected in the Appendix.

Contents

1 The Big Picture: Why Contextual Embeddings? 2
- 1.1 The Problem with Static Embeddings . . . 2
- 1.2 The Journey Ahead . . . 2

2 Text Pre-Processing Pipeline 2
- 2.1 Why Padding? . . . 3
- 2.2 What is Masking? . . . 3

3 Recurrent Neural Networks (RNN) 3
- 3.1 From Feed-Forward to Recurrence . . . 3
- 3.2 The RNN Equation . . . 3
- 3.3 Worked Example: Sentiment Analysis of "I like AI" . . . 4

4 Activation Functions: When to Use Which 5

5 How RNNs Create Contextual Embeddings 5

6 The Vanishing Gradient Problem 6
- 6.1 The Problem . . . 6
- 6.2 Intuitive Example . . . 6

7 LSTM: Long Short-Term Memory 6
- 7.1 The Core Idea . . . 6
- 7.2 The Three Gates . . . 7
- 7.3 Cell State Update . . . 7

8 GRU: Gated Recurrent Unit 8
- 8.1 Motivation . . . 8
- 8.2 The Two Gates . . . 8
- 8.3 Hidden State Computation . . . 8
- 8.4 LSTM vs. GRU at a Glance . . . 9

9 The Complete Evolution: Summary 9
- 9.1 Key Equations Reference Card . . . 9
- 9.2 What Comes Next: Transformers . . . 9

Appendix: Solutions to Self-Check Questions 11

The Big Picture: Why Contextual Embeddings?

The Problem with Static Embeddings

In Lecture 3, we studied **static embeddings** (Word2Vec, GloVe, fastText). They map each word to a *single fixed vector*, regardless of context. Consider the word **"bank"**:

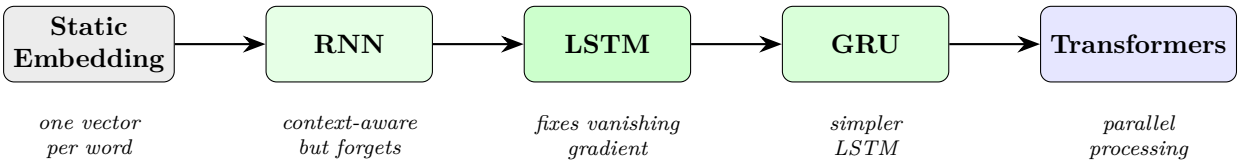
- *"The bank by the river"* — refers to a riverbank (nature)
- *"The bank in the city"* — refers to a financial institution

With static embeddings, "bank" gets **the same vector** in both sentences. The model cannot distinguish between these two meanings.

Key Insight

We need a representation where the **same word gets different vectors** depending on the surrounding words. This is what **contextual embeddings** achieve, and RNNs are the mechanism that makes it possible.

The Journey Ahead



Each architecture solves a limitation of the previous one.

Text Pre-Processing Pipeline

Before any model can process text, raw text must be converted into numerical vectors. The pipeline has **6 steps**:

1. Clean → 2. Stem → 3. Tokenize → 4. Index → 5. Pad → 6. Embed

Worked example: Input = `"I love NLP! Processing123 running"`

Step	Operation	Result
1	Clean (numbers, special chars, lowercase)	`"i love nlp processing running"`
2	Stem (reduce to root forms)	`"i love nlp process run"`
3	Tokenize (split)	`[i, love, nlp, process, run]`
4	Index (map to integers)	`[10, 357, 962, 445, 223]`
5	Pad (fix length to 6)	`[10, 357, 962, 445, 223, 0]`
6	Embed (index → vector)	Each index → dense vector, e.g. dim=4

Why Padding?

Neural networks process data in **batches**. All sequences in a batch must have the **same length**. Padding adds zeros to shorter sequences so they match the longest one. This enables parallel processing ($1\times$ speed) vs. sequential processing ($N\times$ slower).

What is Masking?

Padding tokens are meaningless. **Masking** tells the model to ignore them:

```
mask = (sequence != 0)              # True for real tokens, False for
    padding
output = process(sequence * mask)   # Padded positions contribute
    nothing
```

Without masking, the model wastes computation on zeros and may learn incorrect patterns.

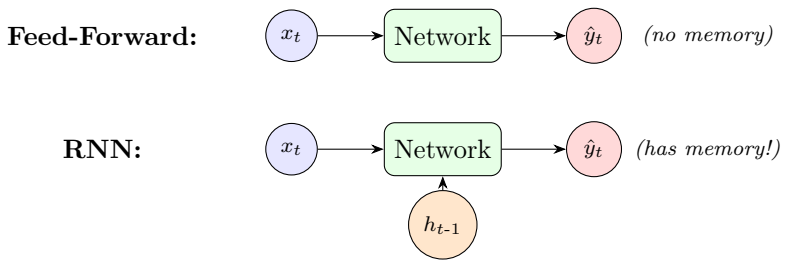
Self-Check Q1

Why can't we simply feed variable-length sequences to an RNN without padding? *(Solution in Appendix)*

Recurrent Neural Networks (RNN)

From Feed-Forward to Recurrence

Feed-forward networks process each input independently—there is no memory of previous inputs. This is fine for tasks like image classification, but language is **sequential**: the meaning of a word depends on what came before.



The key innovation is the **hidden state** h_t, which acts as **memory** passed from one time step to the next.

The RNN Equation

At each time step t, the RNN computes:

$$\boxed{h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)} \tag{1}$$

Symbol	Meaning
$x_t \in \mathbb{R}^m$	Current input (word embedding at time t)
$h_{t-1} \in \mathbb{R}^n$	Previous hidden state (memory from past words)
$W_x \in \mathbb{R}^{n\times m}$	Input weight matrix
$W_h \in \mathbb{R}^{n\times n}$	Hidden state weight matrix (recurrent weights)
$b \in \mathbb{R}^n$	Bias vector
tanh	Squashes output to $[-1, +1]$

Intuition

Think of the hidden state as a person reading a sentence word by word. After each word, they update their mental summary. By the end, h_T contains a summary of the **entire sequence**.

Worked Example: Sentiment Analysis of "I like AI"

Let us trace through an RNN processing the sentence "I like AI" with hidden size = 3.

Setup:

$$W_x = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.2 & 0.3 & 0.1 \\ 0.3 & 0.1 & 0.2 \end{bmatrix}, \quad W_h = \begin{bmatrix} 0.2 & 0.1 & 0.1 \\ 0.1 & 0.2 & 0.1 \\ 0.1 & 0.1 & 0.2 \end{bmatrix}, \quad b = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Word embeddings (dim=3): $x_I = [0.2, -0.1, 0.4]$, $x_{\text{like}} = [0.8, 0.6, 0.3]$, $x_{\text{AI}} = [0.5, 0.4, 0.2]$
Initial hidden state: $h_0 = [0, 0, 0]$

Step 1 — Processing "I"

$$W_x \cdot x_I + W_h \cdot h_0 + b = [0.15,\ 0.25,\ 0.35]$$

$$h_1 = \tanh([0.15,\ 0.25,\ 0.35]) = [-0.05,\ 0.10,\ 0.20]$$

The hidden state now encodes information about the word "I".

Step 2 — Processing "like"

Using $h_1 = [-0.05, 0.10, 0.20]$ as memory:

$$W_x \cdot x_{\text{like}} + W_h \cdot h_1 + b = [0.35,\ 0.25,\ -0.15]$$

$$h_2 = \tanh([0.35,\ 0.25,\ -0.15]) = [0.30,\ 0.20,\ -0.10]$$

Now h_2 encodes information about both "I" and "like".

Step 3 — Processing "AI"

Using $h_2 = [0.30, 0.20, -0.10]$ as memory:

$$W_x \cdot x_{\text{AI}} + W_h \cdot h_2 + b = [0.45,\ 0.15,\ 0.30]$$

$$h_3 = \tanh([0.45,\ 0.15,\ 0.30]) = [0.40,\ 0.10,\ 0.25]$$

The final hidden state $h_3 = [0.40, 0.10, 0.25]$ is a **contextual representation** of the entire sentence "I like AI". This vector can be fed to a classifier to predict sentiment.

Key Point

The **same weights** (W_x, W_h, b) are used at every time step. The RNN is a single cell that gets **unrolled** across time.

Self-Check Q2

In the example above, what would happen to h_3 if we changed the word order to "AI like I"? Would we get the same final hidden state? Why or why not? *(Solution in Appendix)*

Activation Functions: When to Use Which

RNNs and their variants use different activation functions for different purposes:

	tanh	**sigmoid** (σ)	**ReLU**
Formula	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$\frac{1}{1+e^{-x}}$	$\max(0, x)$
Range	$[-1, +1]$	$[0, 1]$	$[0, \infty)$
Think of it as...	Seesaw	Door (open/close)	Bouncer
Used for	Hidden states	Gates (LSTM/GRU)	Deep FFNs, CNNs
Why?	Needs polarity	Yes/no decisions	No upper bound

The Rule of Thumb

- Need to express **positive/negative** values? → Use **tanh** (hidden states)
- Need a **gate** (keep vs. forget)? → Use **sigmoid** (outputs a probability)
- Need **feature extraction** in deep networks? → Use **ReLU** (but rarely in RNNs)

Self-Check Q3

Why do LSTM gates use sigmoid instead of tanh? What would go wrong if we used tanh for the forget gate? *(Solution in Appendix)*

How RNNs Create Contextual Embeddings

This is the payoff of the whole lecture. Consider two sentences processed by the same RNN:

	"The bank by the river"	**"The bank in the city"**
Nature dimension	0.8	0.1
Finance dimension	0.2	0.9
Location dimension	0.6	0.7

The **same word "bank"** ends up with **two completely different vector representations** because the RNN's hidden state carries different contextual information from surrounding words. This is the fundamental difference from Word2Vec/GloVe.

The Vanishing Gradient Problem

The Problem

RNNs learn through **Backpropagation Through Time (BPTT)**. The gradient of the loss with respect to early hidden states involves multiplying the weight matrix W_h many times:

$$\frac{\partial L}{\partial h_t} = W_{hy}^T \frac{\partial L}{\partial y_t} + \sum_{k=t+1}^{T} W_{hh}^T \frac{\partial L}{\partial h_k} \tag{2}$$

If the eigenvalues of W_h are:

- < 1: gradients **shrink exponentially** (vanishing gradient) — the network forgets early words
- > 1: gradients **explode exponentially** (exploding gradient) — training becomes unstable

Intuitive Example

Consider: *"Despite the **cat's** initial fear, the dog became its best **friend**."*

To understand that "friend" relates back to "cat", the model must carry information across many time steps. With vanishing gradients, the influence of "cat" on the prediction at "friend" effectively **disappears**—the gradient signal is too weak to update the weights meaningfully.

Result

Vanilla RNNs work well for **short-range dependencies** but fail for **long-range dependencies**. This is the primary motivation for LSTM.

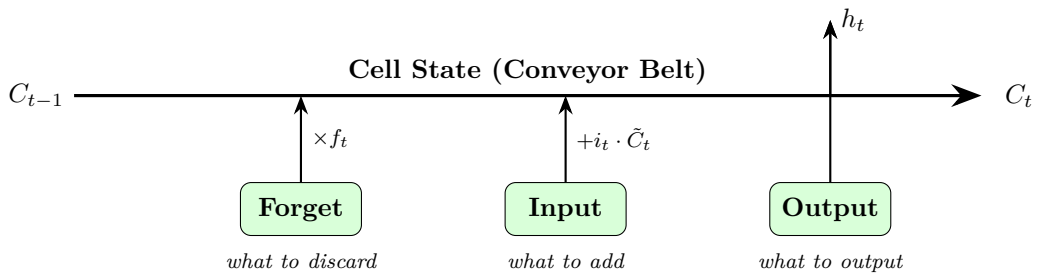
Self-Check Q4

If a vanilla RNN processes a 100-word sentence, can it effectively use information from word 1 when processing word 100? Explain using the vanishing gradient concept. *(Solution in Appendix)*

LSTM: Long Short-Term Memory

The Core Idea

LSTM solves the vanishing gradient problem by introducing a **cell state** C_t—think of it as a **conveyor belt** that runs through the entire sequence. Information can flow along this belt **unchanged**, or be selectively added/removed through **gates**.



The Three Gates

Each gate is controlled by a sigmoid (σ) that outputs values between 0 and 1.

Gate 1: Forget Gate — *"What old information should I discard?"*

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{3}$$

- Output near **0**: forget this information
- Output near **1**: keep this information

Intuition: When reading "The cat sat. **The dog** ran", the forget gate learns to discard information about "cat" when the subject changes to "dog".

Gate 2: Input Gate + Candidate — *"What new information should I store?"*

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \qquad \text{(how much to add)} \tag{4}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \qquad \text{(what to add)} \tag{5}$$

The input gate (i_t) decides *how much* new information to add. The candidate ($\tilde{C}_t$) proposes *what* new information to add. Note: sigmoid for the gate (0/1 decision), tanh for the candidate (can be positive or negative).

Gate 3: Output Gate — *"What part of the cell state should I output?"*

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{6}$$

$$h_t = o_t \cdot \tanh(C_t) \tag{7}$$

The output gate filters the cell state to produce the hidden state h_t.

Cell State Update

The cell state update is the heart of the LSTM:

$$\boxed{C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t} \tag{8}$$

Reading this equation:

- $f_t \cdot C_{t-1}$: Keep the parts of old memory that the forget gate allows
- $i_t \cdot \tilde{C}_t$: Add the parts of new information that the input gate allows

Why This Solves Vanishing Gradients

The cell state C_t is updated through **addition** (not repeated multiplication like in vanilla RNNs). Gradients can flow through the additive path without shrinking, allowing the network to remember information across long sequences.

Self-Check Q5

In an LSTM, what happens to the cell state if the forget gate outputs all 1s and the input gate outputs all 0s? What does this mean in practice? *(Solution in Appendix)*

GRU: Gated Recurrent Unit

Motivation

LSTMs are powerful but **complex**: 3 gates, separate cell state and hidden state, many parameters. GRU simplifies this to **2 gates** and **merges** the cell state and hidden state into one.

The Two Gates

Reset Gate — *"How much past information to ignore?"*

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \tag{9}$$

- $r_t \approx 0$: ignore previous hidden state (fresh start)
- $r_t \approx 1$: fully use previous hidden state

Update Gate — *"How much to keep old state vs. adopt new?"*

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \tag{10}$$

- $z_t \approx 1$: keep previous memory (skip this input)
- $z_t \approx 0$: replace with new candidate

Hidden State Computation

Candidate hidden state:

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1},\ x_t] + b) \tag{11}$$

The reset gate controls how much of the previous hidden state "leaks" into the candidate.

Final hidden state:

$$\boxed{h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t} \tag{12}$$

This is an **interpolation** between the old hidden state and the new candidate. The update gate z_t simultaneously decides what to forget AND what to add—in LSTM, these are separate decisions.

LSTM vs. GRU at a Glance

Feature	LSTM	GRU
Gates	3 (forget, input, output)	2 (reset, update)
States	2 (C_t + h_t)	1 (h_t only)
Parameters	More	Fewer
Training speed	Slower	Faster
Long sequences	Slightly better	Comparable
Small datasets	May overfit	Better generalization

Self-Check Q6

In a GRU, the update gate z_t replaces both the forget gate and input gate of an LSTM. Explain how a single gate can do the job of two. *(Solution in Appendix)*

The Complete Evolution: Summary

Key Equations Reference Card

Model	Core Equation	What it computes
RNN	$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$	Hidden state
LSTM Forget	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	What to discard
LSTM Input	$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	What to store
LSTM Candidate	$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$	Proposed new content
LSTM Cell	$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$	Updated cell state
LSTM Output	$h_t = o_t \cdot \tanh(C_t)$	What to output
GRU Reset	$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$	How much past to use
GRU Update	$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$	Keep old vs. use new
GRU Hidden	$h_t = (1-z_t)h_{t-1} + z_t\tilde{h}_t$	Interpolated state

What Comes Next: Transformers

All RNN-based architectures share three fundamental limitations:

1. **Encoding bottleneck** — the entire sequence is compressed into a fixed-size vector
2. **No parallelization** — tokens must be processed one by one (sequential)
3. **Limited long-range memory** — even LSTMs struggle with very long sequences

Transformers (Vaswani et al., 2017, *"Attention is All You Need"*) solve all three by replacing recurrence with **self-attention**, enabling parallel processing of all tokens simultaneously. This is the subject of Lecture 5.

Self-Check Q7

What is the fundamental architectural difference between RNN-based models and Transformers that allows Transformers to process sequences in parallel? *(Solution in Appendix)*

Appendix: Solutions to Self-Check Questions

Q1: Why can't we feed variable-length sequences without padding?

In theory, a single RNN can handle variable-length sequences because it processes one token at a time. However, in practice, we process data in **batches** for computational efficiency. A batch is represented as a matrix (`batch_size` × `sequence_length`), and matrices require all rows to have the same number of columns. Padding ensures all sequences have uniform length, enabling efficient GPU-based parallel computation across the batch. Without padding, we would need to process each sequence individually (batch size = 1), which is approximately N times slower.

Q2: Would "AI like I" produce the same h_3?

No. The RNN processes words sequentially, and the hidden state at each step depends on the previous hidden state. Changing the word order changes what information the hidden state carries at each step:

- "I like AI": h_1 encodes "I", h_2 encodes "I + like", h_3 encodes "I + like + AI"
- "AI like I": h_1 encodes "AI", h_2 encodes "AI + like", h_3 encodes "AI + like + I"

Even though the same words are processed, the **context accumulation** differs because the hidden state is built incrementally. This is precisely how RNNs capture word order, unlike bag-of-words models.

Q3: Why sigmoid for gates, not tanh?

Gates act as **binary switches**—they decide how much information to keep (1) or discard (0). Sigmoid outputs values in $[0, 1]$, which naturally represents a "percentage" of information to pass through.

If we used tanh (range $[-1, +1]$) for the forget gate, a value of -1 would **negate** the cell state (flip its sign), not forget it. Negative values would **distort** stored information instead of discarding it. Sigmoid ensures the gate behaves like a **valve**: 0 = fully closed, 1 = fully open, 0.5 = keep half.

Q4: Can a vanilla RNN use word 1 at word 100?

Effectively no. The gradient signal from word 100 back to word 1 involves multiplying W_h approximately 99 times. If the largest eigenvalue is, say, 0.9:

$$0.9^{99} \approx 0.00003$$

The gradient shrinks to near zero, meaning updates to the weights based on the relationship between word 1 and word 100 are negligibly small. The network effectively "forgets" word 1 by the time it reaches word 100.

Q5: Forget gate = all 1s, input gate = all 0s?

The cell state update becomes:

$$C_t = 1 \cdot C_{t-1} + 0 \cdot \tilde{C}_t = C_{t-1}$$

The cell state is **passed through unchanged**—no old information is forgotten, and no new information is added. In practice, this means the LSTM is choosing to **completely preserve its current memory** and ignore the current input. This is useful when processing irrelevant tokens (e.g., stop words like "the" or "a") while tracking a long-distance dependency.

Q6: How does one GRU gate replace two LSTM gates?

In LSTM, the forget and input gates operate **independently**. In GRU, the update gate **couples** them:

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$$

- $z_t = 1$: fully adopt new candidate (input=1, forget=0)
- $z_t = 0$: fully keep old state (input=0, forget=1)

The constraint is: **the amount you forget equals the amount you replace**. LSTM can independently control forgetting and adding, while GRU assumes they are complementary. This works well in practice while reducing parameters.

Q7: What allows Transformers to parallelize?

RNNs are inherently **sequential**: computing h_t requires h_{t-1}, which requires h_{t-2}, and so on. Each token must wait for all previous tokens.

Transformers replace recurrence with **self-attention**, which computes relationships between **all pairs of tokens simultaneously** via matrix multiplication. All token representations can be computed **in parallel**, making Transformers dramatically faster on GPU hardware. The trade-off is $O(n^2)$ memory for the attention matrix, but the parallelization speedup far outweighs this cost.